

converts that to a binary and LD links them to the libraries. Now C is high level, and the same C sources can be built on all C compilers. But the assembler is not, and the binaries are absolutely not.

What we need to do is build a version of the GNU toolchain that is capable of producing code meant for execution in Windows.

Enter MinGW32

Binaries also have certain formats. Linux uses the Executable Linkable Format (ELF) binaries. Windows, on the other hand, uses Common Object File Format (COFF) binaries, or technically, a variant of COFF, which is known as Windows PE (Portable Executable). PE files can store executable code, as well as all the resources needed to use that code (that is, pixmaps, icons, audio, animations and what not) all by itself. GCC could always produce COFF binaries, so it was a simple task of patching a few lines of code to make it produce PE files.

GCC versions post v2.95 could produce PE format binaries. With this hurdle cleared, all that remained was a C Runtime Library that would be able to support applications in Windows. A C Runtime Library (CRT) is the library that provides the standard header files and the LibC library. The MinGW Project was thus created, and it published two packages: a CRT that was linked against MSVCRTDLL, or Microsoft's own C runtime library, and an (incomplete but substantial, enough for compiling GNU software) implementation of the Win32API (the Windows Platform SDK headers and libraries) for GCC.

What we are going to do now is build a version of GCC that will produce Win32 binaries.

Downloads

We need to get some source packages: GCC itself, GNU Binutils, w32api and mingwrt. Here are the download links:

- Win32API: <http://nchc.dl.sourceforge.net/sourceforge/mingw/w32api-3.13-mingw32-src.tar.gz>
- MinGW Runtime: <http://nchc.dl.sourceforge.net/sourceforge/mingw/mingwrt-3.15.2-mingw32-src.tar.gz>
- GNU Binutils (Latest snapshot, because release version is broken): <http://sources.redhat.com/mirrors.airband.net/binutils/snapshots/binutils.weekly.tar.bz2>
- GCC 4.4.0 (Yeah, it's been released!): <http://ftp.gnu.org/pub/gnu/gcc/gcc-4.4.0/gcc-4.4.0.tar.bz2>
- GMP: <http://ftp.gnu.org/gnu/gmp/gmp-4.3.1.tar.bz2>
- MPFR: <http://www.mpfj.org/mpfr-current/mpfr-2.4.1.tar.bz2>

Building

It's safest to install something like a compiler to its own prefix, to keep it from mixing up with and fouling up the distro compilers. We will install our copy of the MinGW Cross Compilers to `/opt/mingw`.

First, we need to build the MinGW targeted Binutils.

Open up a terminal, and type the following commands:

```
$ tar -xvf binutils.weeklytar.bz2
$ mkdir build
```

```
$ cd build
$ ./binutils-2.19.51/configure --target=i686-pc-mingw32 --prefix=/opt/mingw
$ make
$ sudo make install
$ export PATH=/opt/mingw/bin:$PATH
```

The last command added the MinGW compilers' bin directory to the PATH variable. Do not exit the terminal. If you do, you'll have to type in the export command again. We'll take care of this later.

Now we need to clean the build directory, and copy the MinGW headers required to build GCC.

```
$ rm -rf * # Remember not to add a forward slash anywhere :)
$ cd ..
$ tar -xvf mingwrt-3.15.2-mingw32-src.tar.gz
$ tar -xvf w32api-3.13-mingw32-src.tar.gz
$ ln -s w32api-3.13-mingw32 w32api
$ sudo cp -r w32api/include /opt/mingw/i686-pc-mingw32
$ sudo cp -r mingwrt-3.15.2-mingw32/include /opt/mingw/i686-pc-mingw32
```

This soft link is required for building the runtime. Now it's time to build GCC. GCC needs to be built in two parts, first a basic C compiler, and then a full set of compilers for all languages.

```
$ tar -xvf gcc-4.4.0.tar.bz2
$ tar -xvf gmp-4.3.1.tar.bz2
$ tar -xvf mpfr-2.4.1.tar.bz2
```

GMP is often miscompiled. But there's nothing that can be done about it, as GMP will be built within GCC itself. To check, you can build GMP and then run a *make check* on it. If you find problems that can be corrected, you'll have to edit the source code yourself.

```
$ cd gcc-4.4.0
$ mv ./gmp-4.3.1 gmp
$ mv ./mpfr-2.4.1 mpfr
$ cd ../build
$ sudo ./gcc-4.4.0/configure --prefix=/opt/mingw --target=i686-pc-mingw32 \
--with-headers=/opt/mingw/i686-pc-mingw32/include \
--disable-shared --enable-languages=c
$ sudo make
$ sudo make install
```

That was the basic compiler build. Notice the use of *sudo* in all the three commands required to build GCC—this is because GCC insists on changing the directory structure in `/opt/mingw` in the configure step itself, and since the makefiles have superuser permissions, we need to have root privileges to do the *make* and *make install*. Also note that we've disabled shared library building; this needs a file called *dllcrt.o* which is part of the MinGW runtime and hasn't been built yet.

We now need to build our C Runtime Library, the Win32API library and the actual build of GCC. First, the runtimes: